

JAVA

Introducción a los conceptos básicos en java



SINTAXIS
BÁSICA



ESTRUCTURAS
DE CONTROL



CLASES Y
OBJETOS



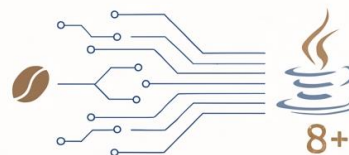
PROYECTOS
REALES

APRENDE
PRACTICA
CREA
PROGRAMA



EL PRIMER PASO
HACIA GRANDES
PROYECTOS

Índice



Introducción	1
A quién está dirigido este libro	1



Módulo 1: Introducción a java y Configuración del entorno

¿Qué es java?	2
La Filosofía WORA: Escribe una vez, ejecuta donde sea	2
La Máquina Virtual de Java (JVM)	3
Pilares de su Arquitectura	3
Herramientas para empezar a programar en java	4
Buenos hábitos en programación	5
Consideraciones importantes antes de usar Java:	6
Bienvenido a java	7

Módulo 2: Fundamentos del lenguaje y Entrada de datos

Tipos de variables.....	10
-------------------------	----



Módulo 3: Estructuras de flujo

If y if else	12
Condicional switch	14
Bucles	15

Módulo 4: Manejo de Errores y Excepciones

Try catch.....	18
----------------	----

Módulo 5: Estructuras de datos lineales y algoritmos

Matrices y vectores	19
Métodos de ordenamiento	20

Módulo 6: Programación Orientada a Objetos (POO)

Objetos y método constructor	21
Que se puede hacer con java	25
Agradecimientos	26



Introducción



Aprender a programar es aprender a pensar con precisión, descomponer problemas complejos en pasos lógicos y construir soluciones capaces de ejecutarse millones de veces sin fatiga. Dentro del vasto ecosistema de lenguajes existentes, Java ocupa un lugar singular: es riguroso, explícito y formativo. Lejos de ocultar los mecanismos que hacen funcionar al software, Java exige comprender qué ocurre con los datos, cómo se organiza la memoria y bajo qué reglas interactúa el código con el hardware.

Este libro, «**Introducción a los conceptos básicos en Java**», nace con un propósito claro: servir como un puente sólido entre el interés inicial por la informática y la comprensión real de la ingeniería de software.

A menudo, quienes dan sus primeros pasos en la programación se encuentran con dos extremos poco pedagógicos: tutoriales superficiales que enseñan a copiar y pegar sintaxis sin explicar el porqué de las cosas, o manuales académicos densos cuya teoría resulta abstracta e intimidante. Este texto busca el equilibrio técnico exacto. Aquí no solo aprenderás a escribir instrucciones válidas para el compilador; comprenderás qué es la Máquina Virtual de Java (JVM), por qué el tipado estricto previene fallos catastróficos en producción y cómo la Programación Orientada a Objetos permite estructurar proyectos a gran escala.



A quién está dirigido este libro

Este material ha sido diseñado para:

- **Estudiantes de ciencias de la computación e ingeniería:** Que necesitan dominar los fundamentos teóricos y prácticos que sustentan las materias de programación y estructuras de datos.
- **Autodidactas y entusiastas de la tecnología:** Que desean construir una base técnica firme y entender el funcionamiento interno del software más allá de la superficie.
- **Desarrolladores principiantes:** Que buscan dar el salto desde lenguajes interpretados o dinámicos hacia un entorno fuertemente tipado y de nivel profesional.

No se asumen conocimientos previos de programación. Comenzaremos desde los cimientos absolutos —la configuración del entorno de trabajo y la anatomía de tu primer programa— hasta avanzar de forma metódica hacia estructuras de control, funciones modulares y los principios esenciales del diseño orientado a objetos.



¿Qué es java?

Módulo 1: Introducción a java y Configuración del entorno



Cuando nos adentramos en el mundo del desarrollo de software, es imposible ignorar la huella de Java. Desde su lanzamiento por Sun Microsystems en 1995, no solo se presentó como un nuevo lenguaje de programación, sino como un cambio de paradigma en la forma en que el software interactuaba con el hardware.

Para entender qué es Java, debemos desglosarlo en dos conceptos fundamentales: es, al mismo tiempo, un **lenguaje de programación** y una **plataforma de ejecución**.

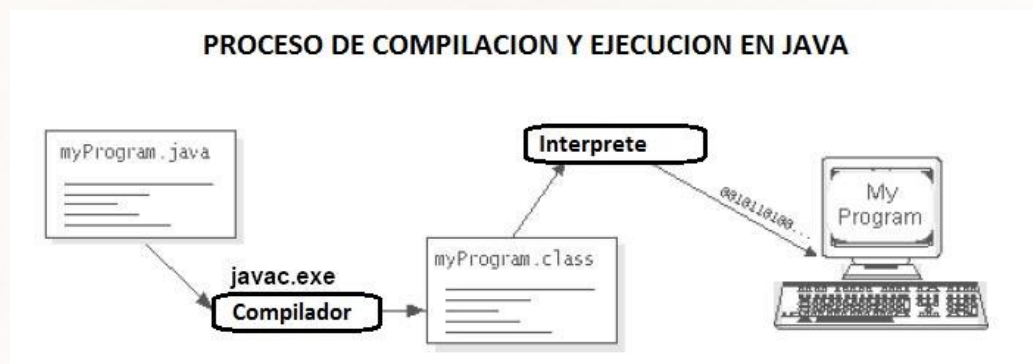
La Filosofía WORA: Escribe una vez, ejecuta donde sea

En los inicios de la informática moderna, los desarrolladores se enfrentaban a un problema logístico masivo. Si escribías un programa en lenguajes como C o C++, el código fuente debía ser compilado —traducido a ceros y unos— específicamente para la arquitectura del procesador y el sistema operativo de destino. Un programa compilado para Windows no funcionaría en Linux ni en macOS sin antes sufrir modificaciones en su código y un nuevo proceso de compilación.



Java nació para resolver esta fragmentación bajo un lema que cambiaría la industria: *Write Once, Run Anywhere* (WORA).

Para lograr esta independencia del hardware, Java introdujo una capa de abstracción brillante. Cuando un desarrollador compila un programa en Java, el código no se traduce directamente al lenguaje máquina del procesador físico. En su lugar, se traduce a un formato intermedio y universal conocido como **Bytecode**.





El Bytecode por sí solo es incomprensible para cualquier procesador convencional. Aquí es donde entra en juego la segunda mitad del ecosistema: la **Java Virtual Machine (JVM)**.

Podemos pensar en la JVM como una "computadora virtual" que reside dentro de nuestra computadora física. Su única función es tomar ese archivo de Bytecode (los archivos .class) y traducirlo en tiempo real a las instrucciones eléctricas exactas que requiere el procesador local.

Esto significa que el programador solo necesita preocuparse por escribir y compilar su código una vez. Mientras el usuario final tenga instalada una JVM en su sistema, el programa se ejecutará de forma idéntica, sin importar si el hardware subyacente es un servidor de alto rendimiento, una computadora portátil de consumo o un teléfono inteligente.

Pilares de su Arquitectura

Además de su portabilidad, la adopción masiva de Java en el ámbito profesional e institucional se debe a tres características técnicas que garantizan su robustez:



1. **Orientación a Objetos Pura:** Java obliga al desarrollador a pensar en términos de "Objetos" y "Clases". En lugar de escribir secuencias de comandos interminables, el código se modulariza en entidades que poseen atributos y comportamientos. Esto permite que los sistemas escalen de manera organizada y facilita el trabajo colaborativo en equipos de ingeniería.
2. **Fuertemente Tipado:** Es un lenguaje estricto que no deja lugar a la ambigüedad. Cada variable debe tener un tipo de dato declarado explícitamente (entero, texto, booleano). Esta rigidez arquitectónica es una red de seguridad que detecta fallos lógicos durante la etapa de escritura, mucho antes de que el programa llegue al usuario final.
3. **Gestión Autónoma de Memoria:** En lenguajes de bajo nivel, el programador debe reservar y liberar memoria RAM manualmente, un proceso propenso a errores fatales como las fugas de memoria (*memory leaks*). Java automatiza esta tarea a través de su Recolector de Basura (*Garbage Collector*), un subproceso que escanea constantemente la memoria para eliminar los objetos que ya no están en uso, optimizando los recursos del sistema de forma invisible.

Hoy en día, casi tres décadas después de su creación, Java sigue siendo la columna vertebral de la infraestructura tecnológica.

Herramientas para empezar a programar en java



Módulo 1: Introducción a java y Configuración del entorno



El programa que utilizaremos para programar en java será **NetBeans**. Para empezar quiero que sepas que NetBeans es un IDE por lo que esto no es más que el lugar donde programaremos, pero podemos hacer esto en cualquier otro IDE. El motivo de usar este programa, es que incluye una amplia variedad de herramientas y una fácil integración con el Lenguaje de Java.

NetBeans:

[Downloading Apache NetBeans 31](#)

<https://netbeans.apache.org/front/main/index.html>



JDK La Java Development Kit es el kit de herramientas básicas que se necesitan para iniciar a programar en java. Tanto su método de instalación como el método de descarga es el mismo que de cualquier aplicación.



JDK:

[Java Downloads | Oracle](#)

<https://www.oracle.com/java/technologies/downloads/>



La página oficial de Java en Oracle es muy útil si se desea aprender más sobre el funcionamiento de cada elemento en java incluyendo librerías, errores, variables etc. Es la documentación oficial de java.

[Overview \(Java Platform SE 8\)](#)

<https://docs.oracle.com/en/java/>



Buenos hábitos en programación



Módulo 1: Introducción a java y Configuración del entorno



Existen buenos hábitos o como muchos le llaman buenas prácticas en al programar en java esos nos ayudan a ser ordenado generar un código limpio, bien estructurado y seguro. Por lo que seguirlas no es una obligación pero sí una recomendación para evitar errores y confusiones al trabajar en equipo o al tratar de modificar el código en el futuro.

1. El primer buen hábito es tener todas las variables principales que se van a usar juntas arriba en el código agrupadas. Otro buen hábito es siempre poner los nombres de las variables en minúsculas. Los métodos deben de estar la inicial en mayúscula.
2. No revisar código generado con IA, esta puede fallar fácilmente y no se revisa puede ser catastrófico.
3. Crear una documentación clara, completa y ordenada es fundamental para que tanto los usuarios como los demás programadores puedan entender el código.
4. Código limpio y legible: Un código limpio y legible es útil para futuros cambios y mejores errores de comprensión.
5. Control de versiones: El control de versiones nos sirve por si llegara a perderse parte de un código o bien este tuviera bugs tener siempre una versión anterior puede ser de gran ayuda.
6. Seguridad: Es importante tener un código seguro para evitar posibles hackeos o algún Malware.





(vuelva a esta sección siempre que vea algún símbolo extraño en el libro)



1. En java existe la diferencia entre usar “=” y doble “==”.
2. El igual “=” se usa para asignar un valor a una variable mientras que “==” se utiliza para comparar. Esto aplica para tipo de variables excepto de tipo texto (String y *char*); con esto se puede comparar si el valor de una variable es igual a otra.
3. Para comparar variables de tipo texto se utiliza el método **equals()** siendo el contenido que esté en el paréntesis al que se va a comparar.
4. Para comparar variables de tipo texto se utiliza el método **equals()** siendo el contenido que esté en el paréntesis al que se va a comparar.
5. Cuando usamos los paréntesis “ () ” en cualquier contexto esto se usa para indicar parámetros cuya función sirven para indicar que cierta parte de un código se ejecutara si y solo si esos parámetros se cumplen. En muchos casos no es necesario usar los parámetros y se puede dejar en blanco.
6. Siempre que escribamos algún texto se debe escribir de la entre comillas dobles “” y en caso de escribir un solo carácter de tipo texto se hace con comillas simples ‘’ esto se aplica siempre.
7. Los operadores lógicos “and” se escriben “&&” el operador “OR” se escribe “||” los operadores “<, >, >=, <=” funcionan igual que en matemática.
8. El uso de “+” este operador funciona para concatenar variables o bien para sumar, por otro lado, “++” Indica que a la variable de tipo entero o decimal se le está sumando 1. Es una abreviatura a (variable = variable +1;), (variable++;) ambos significan lo mismo.
9. El uso de cualquier “-” nos sirve para restar, sin embargo, al igual que ++ existe el “--” el cual resta uno a la variable y se usa de la misma forma que ++; es decir (variable--;)
10. Continue, break, return.



Tips de uso:

Existen muchas otras consideraciones que puede encontrar en la página de la documentación de java en Oracle.

Bienvenido a java

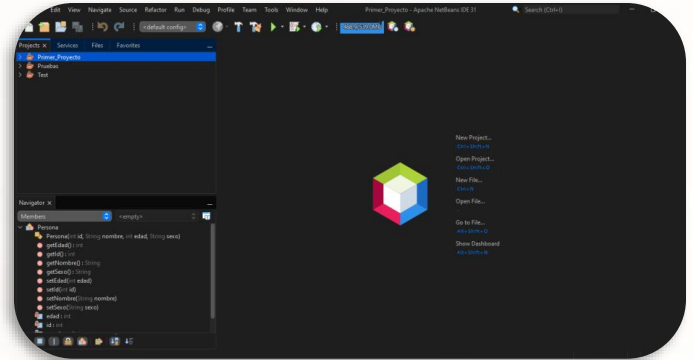


Módulo 1: Introducción a java y Configuración del entorno

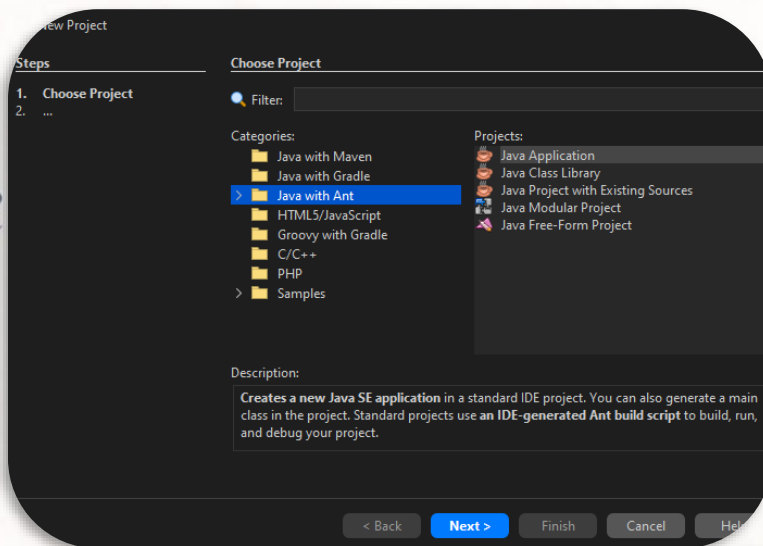


Bien con todo lo anterior visto podremos empezar a usar Java con NetBeans. Pero antes debemos aprender a utilizar el programa de NetBeans.

Al abrir el programa veremos algo de este estilo o parecido. Lo primero es crear un nuevo proyecto seleccionando (New Project) o el símbolo de hoja de papel con un “+” después, tendremos que escoger el tipo de proyecto que queremos crear.

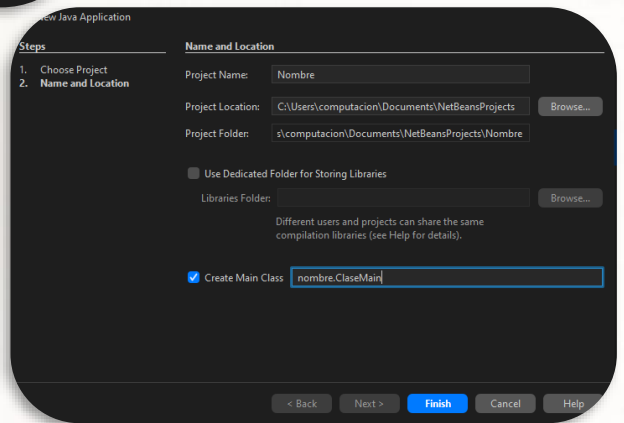


Existen varios, pero se recomienda usar (Java with Ant) y luego (Java application)



Este programa en java es mucho más sencillo de utilizar, también se puede usar (Java with maven) pero esto es solo para personas más experimentadas.

Después de seleccionar (Java Application) se nos habilitará una ventana donde debemos de escoger el nombre de nuestro proyecto (Project Name). Luego de asignar un nombre al proyecto, podremos darle nombre a la clase Main; sin embargo, no será obligatorio hacer esto.



Módulo 1: Introducción a java y Configuración del entorno



Y listo ya tienes u proyecto creado con esto a continuación el significado de cada línea de código que aparece:

```
package nombre;

public class ClaseMain {

    public static void main(String[] args) {

    }

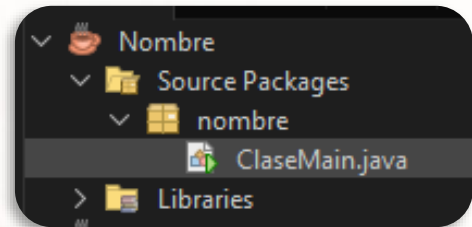
}
```

Línea 1 (package nombre;) significa que esta clase llamada *ClaseMain* pertenece al paquete “nombre” incluido todo su contenido.

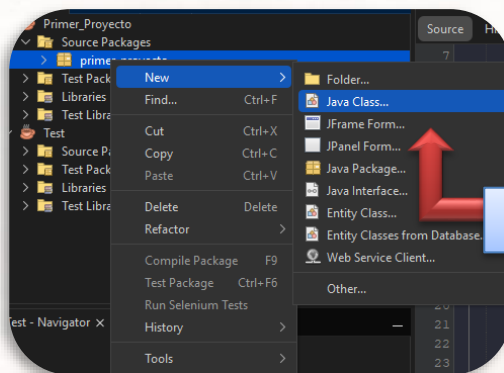
Línea 2 (public class ClaseMain{ }) Es la declaración de una clase. Siempre debe de ir este código pues contiene todo el código dentro de la clase.

Línea 3 (public static void main(String[] args){ }) En

esencia significa que nuestra clase *main* principal está aquí. Siempre que se inicie un proyecto se debe de hacer en la clase *main* por lo que este es el punto de partida de todo programa.

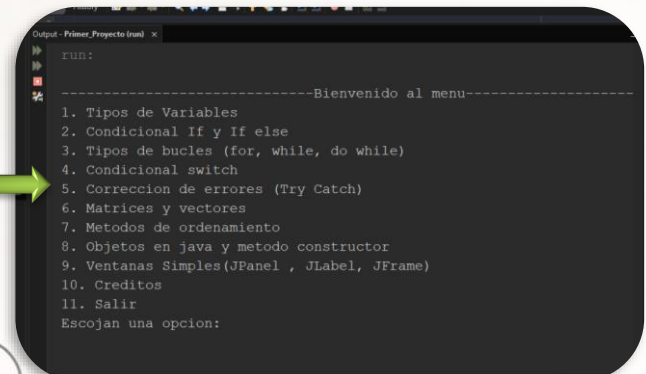


Este es tu proyecto y aquí se encuentra todo su contenido aquí puedes crear clases nuevas dando clic derecho: *New*, *JavaClass*.



Este es el apartado para crear una clase

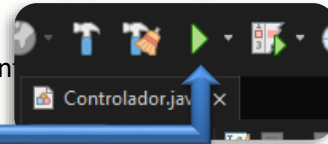
Al ejecutar nuestro código de java podremos ver nuestra salida en la consola.





En este apartado en el triángulo verde podremos ejecutar nuestro proyecto de java.

ulo 1: In



ración del entorno

Para crear un “hola mundo” debemos de usar un **System.out.println()** con esto le estamos indicando a la consola de salida que escriba “Hola mundo”. Existe una variable con **System.out.print()** el cual tiene la diferencia que no realiza un salto de línea después de escribir.

Que es **System.out.println()** este comando le indica al sistema de salida que escriba una línea de texto en la consola.

Scanner: es una de las clases más usadas y útiles que hay en java. Para usar *Scanner* necesitamos crear un objeto de la clase *Scanner* y dentro de este escribir (**System.in**). Esta clase nos sirve para ingresar el valor de alguna variable por medio del usuario. El Scanner no funciona por sí sola si no se importan sus dependencias, que en este caso son (java.util.Scanner) con esto estamos importando las dependencias de la clase Scanner.

La clase **Scanner** al momento de asignar el valor a una variable como en el ejemplo de abajo. Debemos de tomar la variable que asignamos a Scanner y le damos un **nextInt()** esto si es un entero si fuera un texto se usaría **nextLine()** y si es un decimal **nextDouble()**.

```
package nombre;  
  
public class ClaseMain {  
  
    public static void main(String[] args) {  
        System.out.println("Hola mundo");  
    }  
}
```

```
1 package nombre;  
2  
3 import java.util.Scanner;  
4  
5  
6 public class ClaseMain {  
7  
8     static Scanner entrada = new Scanner(System.in);  
9  
10  
11 public static void main(String[] args) {  
12  
13     int variable = 0;  
14  
15     variable = entrada.nextInt();  
16  
17 }  
18  
19 }
```

Estas son los primeros conceptos fundamentales que se usaran la mayor parte del tiempo al programar en java.

Con esto ya podremos empezar a programar en java y entender su funcionamiento.

Tips de uso:

Para aprender java no solo basta con ejemplos es importante realizar ejercicios.

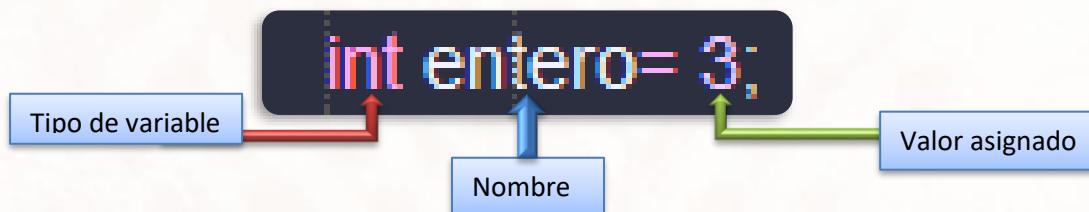


Tipos de variables

Módulo 2: Fundamentos del lenguaje y Entrada de datos

Las variables se clasifican en dos tipos:

Al crear una variable, esta debe de tener la siguiente estructura:



Tipos de variables: Es el tipo a la que pertenece la variable (int, String, Double, etc.).

Nombre: Una variable se le puede tener asignar cualquier nombre.



Valor Asignado: El valor que tiene la variable.

Los tipos de variables:

int:

La variable de tipo *entero* solo admite números enteros:

```
//Variable de tipo entero admiten solo numeros enteros  
int entero = 3;
```

String:

Un tipo de variable que admite y guarda cualquier tipo de texto. Por regla general cualquier tipo de texto se debe de escribir entre comillas dobles ("") como se ve en el ejemplo de abajo.

```
//La variable String siempre es un texto  
String texto1 = "Hola soy un texto";
```

boolean:

Las variables de tipo *booleano* son variables que solo pueden admitir dos estados (*true* o *false*).

```
//Los valores booleanos siempre son true o false  
boolean booleano = true;
```



Módulo 2: Fundamentos del lenguaje y Entrada de datos

double:

Las variables *double* son variables decimales las cuales pueden admitir tanto números enteros como decimales.

```
//La variable Double es para decimales  
double decimal = 3.1416;
```

char:

Las variables de tipo *char* son aquellas que pueden guardar un solo carácter de tipo texto. Esta variable se guarda en comillas simples.

```
char character = 'q';
```



Las variables se pueden concatenar con otros variables del mismo tipo. Al momento de hacer esto automáticamente la variable que se use se convierten a la variable que se asignen. Esto no aplica para todos los tipos de variable.

```
String texto = "Encadenando";  
String texto2 = "texto";  
String cadena = texto + texto2;
```

Aquí se puede apreciar de mejor forma como tomamos dos variables de tipo *String* y las encadenamos, si imprimiéramos la variable **“cadena”** nos aparecería el contenido de la variable **“texto”** y **“texto2”** junto.

El uso de una variable correcta ahorra recursos y tiempo. En un programa.

Tips de uso:

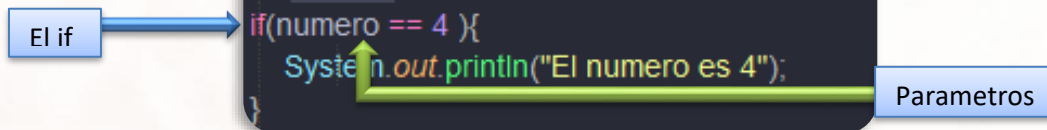
Se pueden crear variables sin darles un valor. Para esto no debemos de poner “=” y su valor.

If y if else

Módulo 3: Estructuras de flujo

El *if* es un condicional que funciona como un booleano que válida si los parámetros son verdaderos o falsos.

Estructura de un *if*:



if: Es la palabra que usaremos para empezar un *if* funciona igual que la creación de una variable.

Parametros: Los parámetros son los que utilizaremos para validar las condiciones que se deben de cumplir para que este funcione. Pueden ser una o varias condiciones.

Else: Este es una medida de prevención. Si las condiciones del *if* no se cumplen en entonces inmediatamente se dirige a la parte de *else*.

If: El *else* no es obligatorio que acompañe a un *if*. Esto en java se toma como si no se cumple la condición simplemente no hace nada.

```
int numero = 4;
if(numero == 4 ){
    System.out.println("El numero es 4");
}
System.out.println("El programa continua");
```

En este ejemplo se valida si la variable “numero” es 4 en. Si esta afirmación es verdadera la consola nos escribirá “El numero es 4” en caso contrario el programa continuara y no realizara ninguna acción.

If else: El *if else* tiene la función que si no se cumple la condición se realizará determinada acción.

```
int numero = 4;
if(numero == 4 ){
    System.out.println("El numero es 4");
}else{
    System.out.println("El numero no es 4");
}
System.out.println("El programa continua");
```

En este ejemplo realizamos la misma validación pero esta vez en caso de que no se cumpla la condición inicial se realizara se escribirá “El numero no es 4”.



If else + else if: El *else if* es una segunda condicional siendo que si no se cumple la primera se valida la segunda y si esta no se cumple entonces se valida la tercera y así sucesivamente hasta llegar al *else* donde de lo contrario no se valida ninguna de las anteriores.



Módulo 3: Estructuras de flujo

```
int numero = 4;
if(numero == 4){
    System.out.println("El numero es 4");
}else if(numero == 3){
    System.out.println("El numero es 3");
}else{
    System.out.println("El numero no es 4 ni 3");
}
System.out.println("El programa continua");
```

En este caso podemos apreciar como si no se cumple la primera condición se valida la siguiente en orden hasta que se cumpla una. Y en caso de no cumplirse alguna se dirige al **else** por defecto. En un **if** solo se puede cumplir una condición por lo que si se cumple el **if** o algún **else if** inicial ya no se ejecutará el resto de **else if** que sigan.



Dentro de un **if** podemos usar cualquier tipo de condición incluso cualquier tipo de variable (int, String, boolean, etc.).



Tips de uso:

Si es necesario que se comprueben varias condiciones simultáneamente es útil usar los “&&” y “||”.

Condicional switch

Módulo 3: Estructuras de flujo

El condicional *switch* toma una variable y válida diferentes casos para esa variable puede ser cualquier tipo de variable (*int*, *String*, *doublé*, etc.) el objetivo es validar diferentes casos para una misma variable.

Ejemplo de aplicación para el condicional Switch:

```
switch(numero){  
    case 1:  
        System.out.println("Eres el 1");  
        break;  
    case 2:  
        System.out.println("Eres el 2");  
        break;  
    case 3:  
        System.out.println("Eres el 3");  
        break;  
    case 4:  
        System.out.println("Eres el 4");  
        break;  
    case 5:  
        System.out.println("Eres el 5");  
        break;  
    //El default nos indica el caso donde si ninguna de las anteriores se cumple entonces  
    //se hara la siguiente instruccion  
    default:  
        System.out.println("No eres un numero del 1-5");  
}
```

En este caso estamos valuando el valor de la viable “numero” donde la estructura funciona de la siguiente manera:

Case (lo que se busca):

Él case es el valor que buscamos dentro de la variable numero si esta variable fuera de tipo texto se podría colocar letras o texto en lugar de números. Recordando que el texto siempre se debe de estar entre comillas.

Ejemplo con una variable de tipo texto:

Default: Es el resultado por defecto que se usa si no se encuentra ninguno de los resultados de los *case* que contiene el *switch* se dirige al *default* la cual limitamos a ejecutar instrucciones simples.

```
String texto = "soy un texto";  
switch(texto){  
    case "soy un texto":  
        System.out.println("texto 1");  
        break;  
    case " prueba":  
        System.out.println("texto 2");  
        break;  
    default:  
        System.out.println("si no se cumple ningun caso");  
}
```

Tips de uso:

El uso del **break;** es indispensable para evitar el que el switch ejecute todos los case te animo a que pruebes el mismo ejemplo de switch sin break y veas el resultado.



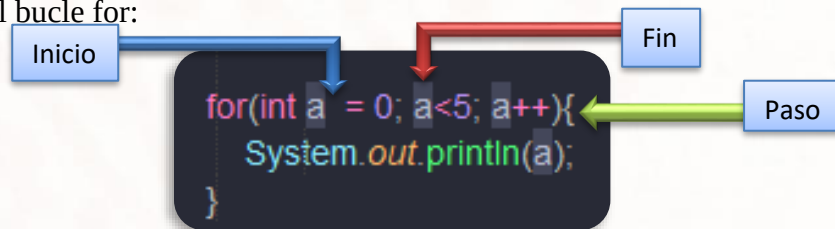
Bucles

Módulo 3: Estructuras de flujo

Los bucles son como su nombre dice una acción que se repetirá hasta que se cumplan una condición. Existen varios tipos de bucles principales.

For: El bucle for se repite para cierta variable mientras se cumpla una condición este bucle es cíclico y se realiza de forma secuencial. Normalmente, se usan variables de tipo entero, pues es más sencillo usar este bucle con este tipo de variable.

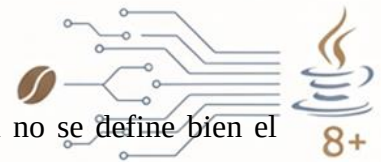
Estructura del bucle for:



Parámetros: Los parámetros en el bucle for se dividen en 3 partes: inicio, fin, paso. El inicio será la variable donde empezara el bucle, El final será la condición para determinar cuándo terminará el bucle, Por otro lado el paso será la velocidad como se ira avanzando el bucle.

No es obligatorio que la variable de recorrido que se usa se inicialice dentro del **for**. Se puede usar alguna otra variable perfectamente sin restricciones. Por convicción se suele inicializar las variables dentro del bucle para evitar saturar el sistema.

```
-----Bucle para (for)-----
System.out.println("Bucle For");
for(int a =1; a<=3; a++){
    //si el valor de a es menor o igual a 3 entonces se ejecutara el siglo para mientras la consicion se cumpla
    System.out.print(a + " ");
}
```

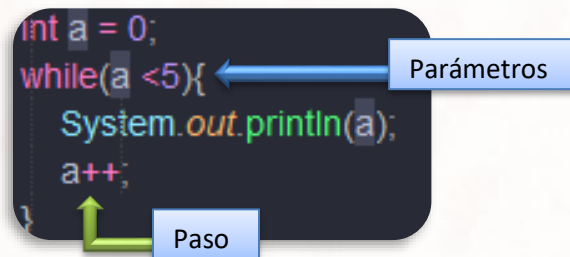


While: El bucle while se repite mientras se cumpla su condición. Si no se define bien el parámetro donde deba terminar el programa este se repetirá infinitamente.



Módulo 3: Estructuras de flujo

Estructura del bucle while:



Parámetros: Para este bucle su único parámetro indica cuando debe detenerse.

Paso: A diferencia del bucle for en este bucle no existe un parámetro para el paso por lo que nosotros mismos debemos de hacer la condición de como deberá de avanzar el bucle.

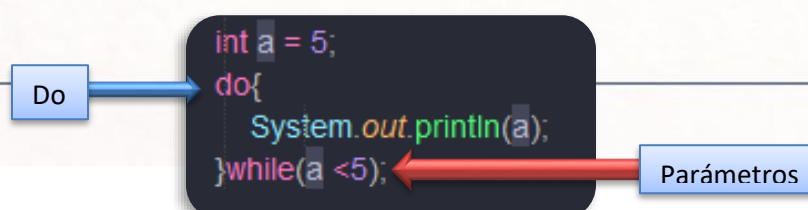


El bucle while(mientras) es muy utilizado cuando no sabemos el límite de una instrucción compleja. Este bucle se ejecutará mientras la condición se cumpla por lo que si se cumple la condición antes de que se inicie el bucle este no se ejecutara y continuara el programa. Pero si no realizamos un Paso dentro del bucle este se ejecutará infinitamente.

```
//El while se puede ejecutar una o ninguna vez
System.out.println("Bucle While");
while(entero < 5){
    System.out.print(entero + " ");
    entero++;
}
```

Do while: Este bucle funciona exactamente de la misma forma que el bucle **while** con la diferencia que este bucle SIEMPRE se ejecutara por lo menos una vez. Incluso si la condición no se cumple.

Estructura del bucle Do while:





Módulo 3: Estructuras de flujo

Do: El Do es la parte del código que se realizara por lo menos una vez esto significa que se ejecuta antes del While.

Parámetros: Los parámetros funcionan y son exactamente los mismos que los del bucle while.

Ejemplo de bucle do while:

```
//El do while siempre se ejecutara una vez
int entero = 2;
//mientras entero sea menor a
System.out.println("Bucle Do While");
do{

    System.out.print(entero + " ");

    entero++;

}while(entero < 10);
```



En el bucle *Do while*(hacer mientras) podemos notar como el bucle se repetirá mientras la variable “entero” sea menor a 10, pero si se llegara a cambiar este parámetro a 2 que es el valor inicial de la variable entero se ejecutará una vez el comando aunque sabemos que la condición de $2 < 2$ no es verdadera.

Return: Este finaliza el método antes que se termine de ejecutar el resto. Es perfecto para momentos donde no queremos que se siga ejecutando el resto del código. Como bucles, ordenamientos, búsquedas entre otros.

Continue;

Break: El break finaliza la parte de código que se esté ejecutando en el momento más no termina un método. Por ejemplo nos puede ayudar a terminar un for pero al hacerlo el método continúa avanzando.



Tips de uso:

Utiliza el bucle for cuando algo se va de repetir muchas veces y el bucle while cuando necesites una respuesta concreta



Try catch

Módulo 4: Manejo de Errores y Excepciones

Este nos ayuda a encontrar errores y prevenirlos para evitar que el código colapse. Es de vital importancia utilizar lo para corregir una amplia variedad de errores.

Estructura de un try catch:

```
int entero;
System.out.println("Escribe una letra para la variable entero");
try{
    entero = dato.nextInt();
}catch(Exception e){
    System.out.println("El valor no es un numero entero");
}
dato.next();
//Bien ahora que pasa si no usamos el try catch
System.out.println("Perfecto ahora vuelve a escribir una letra");
entero = dato.nextInt();
//El programa no lo pudo procesar y colapsa
```



El try catch se conforma de 2 partes, la parte *try* que esta realiza el código que trataremos de evitar errores, y la parte *catch* que se encarga de capturar el código y realizar alguna o ninguna acción en caso de haber algún error.

Dentro del *catch* tenemos un **Exception** esta es el error que evitaremos. Existen muchos tipos de excepciones por practicidad de este ejemplo se usó **Exception** el cual es la librería general que engloba todos los tipos de excepciones que existen. Esto no significa que sea lo más práctico de usar.

El try catch es obligatorio usarlo siempre que se le pida al usuario escribir algún valor para alguna variable.

Tips de uso:

Siempre usa el try catch cuando necesites que el usuario di jite algún valor esto evitara una gran cantidad de errores.



Matrices y vectores

Módulo 5: Estructuras de datos lineales y algoritmos

Las matrices y vectores tienen la función de guardar más de un dato dentro de ellas con el propósito de buscarlos y poder seleccionarlos después incluso modificarlos o borrar uno sin borrar todos los datos guardados.

Vectores:

```
//Vectores
int[] vector = {10, 4, 2, 2, 6};
//Los vectores son matrices de 1x1
System.out.println("Ejemplo de un vector");
for(int a = 0; a<=4; a++){
    System.out.print(vector[a] + " ");
}
```



El vector es lineal y sus datos se guardan en 1x1 por lo que solo podemos guardar 1 dato por cada fila y columna.

Para crear un vector es necesario que a la par de la variable se coloque “[]” las cuales indican que será un vector. Las clases también pueden tener vectores.

```
Persona[] persona = new Persona[100];
```

En este ejemplo estamos creando un vector para la clase **Persona** darnos cuenta de que este vector tiene un tamaño de 100 espacios por lo que solo podremos guardar 100 elementos.

Los vectores siempre empiezan a escribirse desde la posición 0 por lo que para nosotros la posición 1 es en realidad la posición 0 en el vector, eso quiere decir que la posición n será la posición n-1 para la matriz o vector.

Matriz: En la matriz los datos se guardan en una posición de [i][j] por lo que cada fila tendrá j-1 columnas y i-1 filas. En el ejemplo para crear una matriz no difiere mucho de un vector salvo que está lleva doble “[]” y al momento de guardar datos debemos de escoger en cuál fila y columna hacerlo.

```
int[][] matriz = { {1, 2, 3}, //fila 0
                   {4, 5, 6}}; //fila 1

for(int a=0; a<matriz.length; a++){
    for(int c=0; c<matriz[a].length; c++){
        System.out.print(matriz[a][c] + " ");
    }
    System.out.println("");
}
```

Métodos de ordenamiento

Módulo 5: Estructuras de datos lineales y algoritmos



Los métodos de ordenamiento consisten en tomar un vector o una matriz y ordenar sus elementos. Existen varios métodos de ordenamiento, cada uno con ciertas ventajas y desventajas por simplicidad se enseñará como aplicar el método de ordenamiento de Burbuja el cual es el más sencillo de usar, pero el menos eficiente.

Estructura del método de ordenamiento de burbuja:

```
//Lo empezamos a ordenar
for(int i = 0; i<=4; i++){
    for(int j = 0; j<=3; j++){
        if(matrix[j] > matrix[j+1]){
            bandera = matrix[j];
            matrix[j] = matrix[j+1];
            matrix[j+1] = bandera;
        }
    }
}
```



El método de la burbuja **utiliza 2 bucles for** y una variable **auxiliar** que su función es ayudarnos a hacer el movimiento. En el ejemplo la variable auxiliar la llamamos bandera.

Bien esa es la forma base de usar el ordenamiento por burbuja ahora veremos un ejemplo más práctico

En este ejemplo además de realizar el ordenamiento de burbuja se imprimen los pasos cada movimiento del vector *matrix* con esto podemos entender cómo funciona este método de ordenamiento. Su función es obtener el valor actual y comprobar si es mayor al anterior y en caso de ser menor los movería realizando esta comparación realizando esto las veces que sean necesarias hasta tener el vector ordenado.

Los métodos de ordenamiento nos ayudan a tener todos nuestros datos ordenados para una fácil modificación y visualización de estos.

```
for(int i = 0; i<=4; i++){
    for(int j = 0; j<=3; j++){
        if(matrix[j] > matrix[j+1]){
            bandera = matrix[j];
            matrix[j] = matrix[j+1];
            matrix[j+1] = bandera;
            //Esta parter no es obligatoria solo para ver c
            System.out.print(" ");
            for(int k=0; k<matrix.length; k++){
                System.out.print( matrix[k] + " ");
            }
            System.out.println(" ");
            //Esta parter no es obligatoria solo para ver c
        }
    }
}
```



Objetos y método constructor

Módulo 6: Programación Orientada a Objetos (POO)



La **Programación Orientada a Objetos (POO)** es el paradigma central sobre el que se construye todo el ecosistema de Java. En lugar de escribir instrucciones secuenciales aisladas, la POO nos obliga a estructurar nuestro software organizando el código en **clases y objetos** que representan entidades del mundo real mediante atributos (características) y métodos (comportamientos).

Para diseñar sistemas robustos, legibles y escalables a gran escala, el paradigma de la POO se sostiene sobre **cuatro pilares fundamentales: Abstracción, Encapsulamiento, Herencia y Polimorfismo.**

1. Abstracción

La **Abstracción** consiste en identificar las características esenciales de una entidad del mundo real e ignorar los detalles complejos o irrelevantes para nuestro sistema.

Al abstraer, definimos qué datos necesita manejar una **clase** para cumplir su propósito dentro del programa:

Atributos: Representan el estado o las características de la entidad.



Métodos: Representan las acciones o funcionalidades que puede ejecutar.

2. Encapsulamiento

El **Encapsulamiento** es el mecanismo que oculta el estado interno de un objeto y restringe el acceso directo a sus atributos desde otras clases. Esto protege los datos contra modificaciones no autorizadas o inconsistentes.

Para lograr un encapsulamiento correcto en Java se aplican tres reglas principales:

Declarar los atributos como **privados (private)**.

Proporcionar un **método constructor** para inicializar el objeto utilizando la palabra reservada **this**.

Permitir el acceso y modificación controlada mediante **métodos accesorios: Getters (getName)** para leer el valor y **Setters (setName)** para asignarlo o validarlo.

3. Herencia

La **Herencia** es la capacidad de crear una nueva clase (**subclase** o clase hija) a partir de una clase existente (**superclase** o clase padre). La clase hija hereda los atributos y métodos de la clase padre, lo que evita la duplicación de código y fomenta la **reutilización de código**.

En Java, la herencia se establece utilizando la palabra reservada **extends**.



4. Polimorfismo



El **Polimorfismo** (que significa "*muchas formas*") permite que objetos de distintas clases respondan al mismo llamado de un método, pero ejecutando comportamientos diferentes según la clase específica a la que pertenecen.

En Java, el polimorfismo se implementa principalmente a través de la **sobrescritura de métodos** (usando la anotación **@Override**), permitiendo que una subclase redefina el comportamiento de un método heredado de su superclase.

Cuadro Resumen de los Pilares de la POO

Pilar	Concepto Clave	Palabra Clave / Herramienta en Java	Beneficio Principal
Abstracción	Modelar solo los rasgos esenciales.	Clases, Atributos y Métodos.	Simplifica la complejidad del sistema.
Encapsulamiento	Proteger datos y restringir acceso directo.	private , this , getters y setters .	Aumenta la seguridad y evita corrupción de datos.
Herencia	Crear clases nuevas basadas en clases existentes.	extends , super .	Elimina código duplicado y promueve la reutilización.
Polimorfismo	Ejecutar un mismo método con múltiples comportamientos.	@Override , sobrescritura de métodos.	Otorga flexibilidad y dinamismo al código.



Como crear un objeto:

```
Scanner dato = new Scanner(System.in);
Persona[] persona = new Persona[100];
Controlador opciones = new Controlador();
```

En esencia cada vez que usamos la palabra **new** estamos generando un objeto. Los objetos nos ayudan a crear instancias que dependiendo que haya dentro de ese objeto podemos hacer todo tipo de cosas.

Para hacer el llamado de un objeto se usa una instancia

Ejemplo de un llamado de un objeto:



```
opciones.tiposVar();
```

Aquí estamos llamando el objeto `opciones` de la clase `Controlador` e instanciamos el método `tiposVar()`. Por lo que creamos un enlace directo a este fragmento de código de esta clase

Para crear el método constructor debemos crear una clase con el nombre que le daremos seguido colocar algunas variables para este, las variables no son necesarias que se inicialicen.

```
public class Persona {  
  
    private int id;  
    private String nombre;  
    private int edad;  
    private String sexo;  
  
    Persona(int id, String nombre, int edad, String sexo){  
        this.edad = edad;  
        this.id = id;  
        this.nombre = nombre;  
        this.sexo = sexo;  
    }  
}
```



Ahora si podemos crear el método constructor el cual nunca será de tipo `void` ni tampoco de tipo `int`, `String` o algún otro. Es un método especial

Estructura del método constructor:

```
Persona(int id, String nombre, int edad, String sexo){  
    this.edad = edad;  
    this.id = id;  
    this.nombre = nombre;  
    this.sexo = sexo;  
}
```

Si nos damos cuenta en sus parámetros definimos en qué orden y de qué tipo queremos que esté el método. Esto nos sirve por si necesitamos más de un método constructor incluso si se necesita un método con 0 parámetros el cual nos podría servir para hacer llamadas. O alguna otra interacción.

Dentro de este método usaremos el **this** previamente explicado su función este. Con esto estamos asignando un valor a la variable creando una instancia de este con uno de los parámetros del método constructor.



Módulo 6: Programación Orientada a Objetos (POO)



Setters y Getters: Los setters nos ayudan a asignar un valor o a cambiar el valor que ya se tenía dentro de una matriz. Mientras que los Getters nos ayudan a obtener el valor que hay dentro de un objeto.

Estructura de un setter y un getter.

Los getters deben ser métodos con el tipo de la variable que se solicita y retornar la variable solicitada. Los setters deben de ser de tipo void y tener un parametro que reciba el tipo de variable donde se establece el nuevo valor mediante un this

```
public int getId() {  
    return id;  
}  
  
public void setId(int id) {  
    this.id = id;  
}
```

Estos conceptos de la programación orientada a objetos son importantes aprenderlos y se pueden aplicar fácilmente. Estos conceptos quedaran mejor explicados en la siguiente edición de este libro. Con ejemplos incluido.



A partir de ahora ya puedes empezar a crear tus propios proyectos de aplicación en java, puedes usar cualquier programa de programación y realizar cualquier reto que te propongas. Acompaña todo esto con todas las herramientas y documentación que este libro te ha proporcionado para ayudarte a crear cualquier proyecto.

Tips de uso:

Los setters y getters se recomiendan seguir la estructura de setName donde Name es el nombre de la variable lo mismo para los getName. Esto facilita el uso de los setters y getters



Que se puede hacer con java

Módulo 7: Ecosistema java y Sigüientes Pasos



En NetBeans con Java se pueden hacer todo tipo de cosas, pues este es un programa profesional con el que podemos crear aplicaciones, videojuegos, servidores, API, etc.

El límite solo es tu imaginación y recuerda que la programación se puede usar para crear todo tipo de programas simples y complejos:

Ahora como ultimo consejo es que no se quedes solo con este libro busca más información avanzada y crea tus propios proyectos en java. Pueden ser cosas simples el objetivo es aprender la lógica de la programación y como con algo simple puedes crear cosas enormes.

Como ultimo

Todo el código y documentación está en este repositorio de github:

https://github.com/Victor482066/JAVA_introduccion_conceptos_java.git



Proyecto sorpresa sobre el potencial que tiene java:

En este mismo repositorio encontrara un archivo .exe el cual tendrá un pequeño proyecto de programación real hecha en java, como motivación para que tu te propongas a crear tus propios proyectos.



Agradecimientos

Este libro nació con la firme convicción de democratizar el acceso al conocimiento y ofrecer una enseñanza libre, accesible y de calidad en la programación en Java para las nuevas generaciones de estudiantes.



Expreso mi más sincero agradecimiento al **Instituto Técnico Vocacional Privado Amatitlaneco**, institución cuya visión e iniciativa hicieron posible el desarrollo y culminación de este proyecto educativo. Su compromiso con la formación técnica y el desarrollo integral de los jóvenes ha sido el motor fundamental detrás de cada página de esta obra.

Asimismo, agradezco a los docentes, compañeros y mentores que brindaron su tiempo, retroalimentación y apoyo incondicional durante el proceso de elaboración de este material, así como a los estudiantes, cuya curiosidad y ganas de aprender le dan verdadero sentido a este libro.

